Carbon-Aware Deep Reinforcement Learning Platform

Risk-Aware Deep Reinforcement Learning for Sustainable Workload Scheduling in Geo-Distributed Data Centers



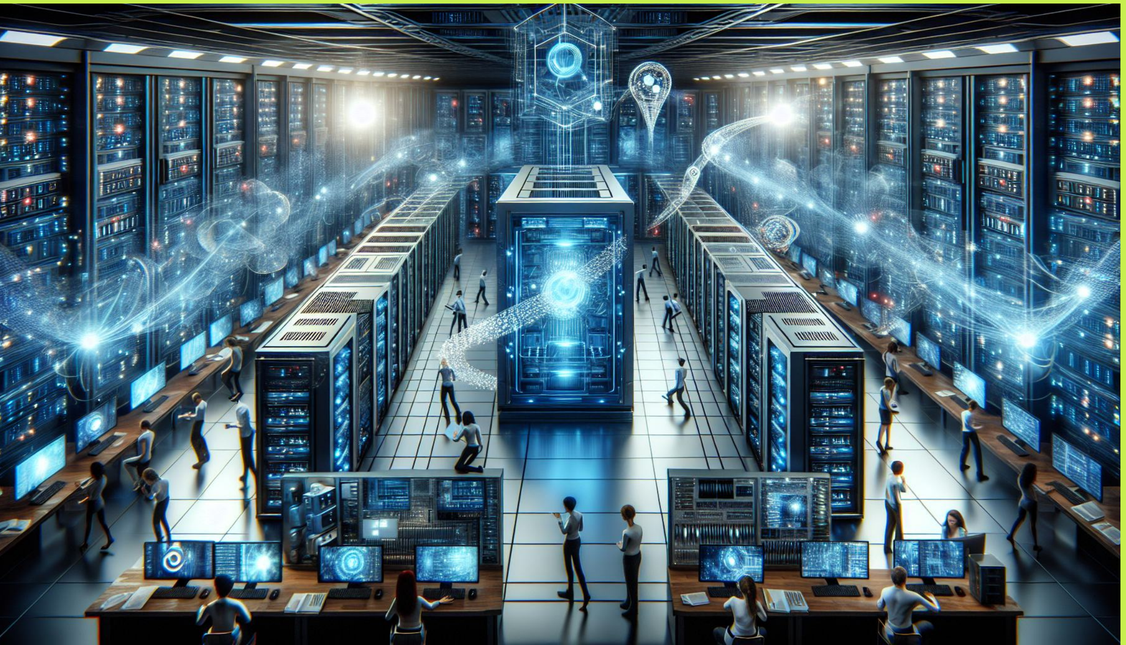
This is a production-grade system that uses Deep Reinforcement Learning to minimize the carbon footprint of distributed computing workloads without compromising performance or reliability.

Recurrent PPO agent with LSTM memory

Neural Storm
Devpos Hackaton

The Problem - Not All Electrons Are Equal

We often think of electricity as a uniform commodity. It isn't. A workload running in a wind-heavy grid at night emits a fraction of the carbon of the exact same workload running on a fossil-heavy grid at sunset. Most schedulers ignore this variability entirely.



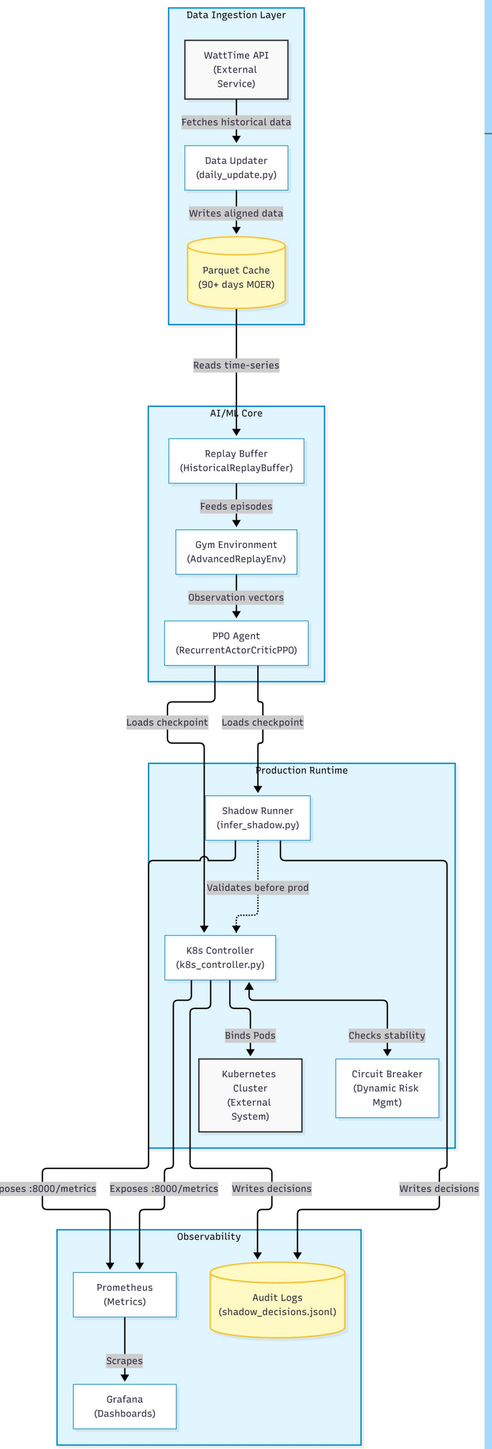
The Data Center Carbon Challenge

1. Stat: Data centers consume ~2% of global electricity (projected to rise with AI).

2. The Reality: Grid carbon intensity varies by 14 x across time and geography.

3. Examples:
   - Iowa at 3 AM (Wind) = 50 gCO_2e/kWh
   - Texas at 6 PM (Peak Demand) = 700 gCO_2e/kWh

4. The Missed Opportunity: When and where you compute matters just as much as how efficiently you compute.



The Scheduling Challenge

A Complex Multi-Objective Optimization Problem

THE CONSTRAINTS (THE «RULES»):

1.SLA Compliance:

Interactive tasks need low latency.

2.Migration Costs:

Moving data/tasks costs carbon and time.

3.Capacity Limits:

Don't overload any single node.

4.Uncertainty:

Grid conditions change constantly; forecasts are imperfect.

This isn't just about picking the greenest node. It's a high-dimensional, non-stationary optimization problem. If we blindly move tasks to save carbon, we might violate SLAs or crash nodes. We need an intelligent agent that can balance these competing objectives in real-time.



The Solution: Carbon-Aware Deep RL Platform



Core Concept: A Kubernetes-native controller powered by Recurrent Deep Reinforcement Learning (PPO + LSTM).

KEY CAPABILITY

Temporal & Spatial Shifting:

Learns to schedule tasks in the right place at the right time.

Implicit Forecasting:

Uses LSTM memory to learn grid trends without relying on perfect external forecasts.

Enterprise-Grade Safety:

Built-in risk management and circuit breakers.

SPATIAL DELTA

~435 g CO_2 e/kWh

PREDICTIVE POWER METRICS

$R^2 = 0.22$ at 3-Hour Horizon

SLA COMPLIANCE RATE

99.20%

Our platform solves this by deploying a custom Kubernetes controller. Instead of hard-coded rules, it uses a Recurrent PPO agent - an AI that learns the complex dynamics of the power grid through experience, while strictly adhering to operational safety constraints.

High-Level Architecture

This is the C4 model system architecture (Container Diagramm). It's modular and production-ready. The data pipeline keeps our grid data fresh. The AI core handles the heavy lifting of training. Crucially, we separate the «Shadow Mode» runner - which validates decisions safely - from the actual Kubernetes Controller that binds Pods to nodes.

Key Components

DATA PIPELINE:

Ingests real-time WattTime MOER data

AI CORE:

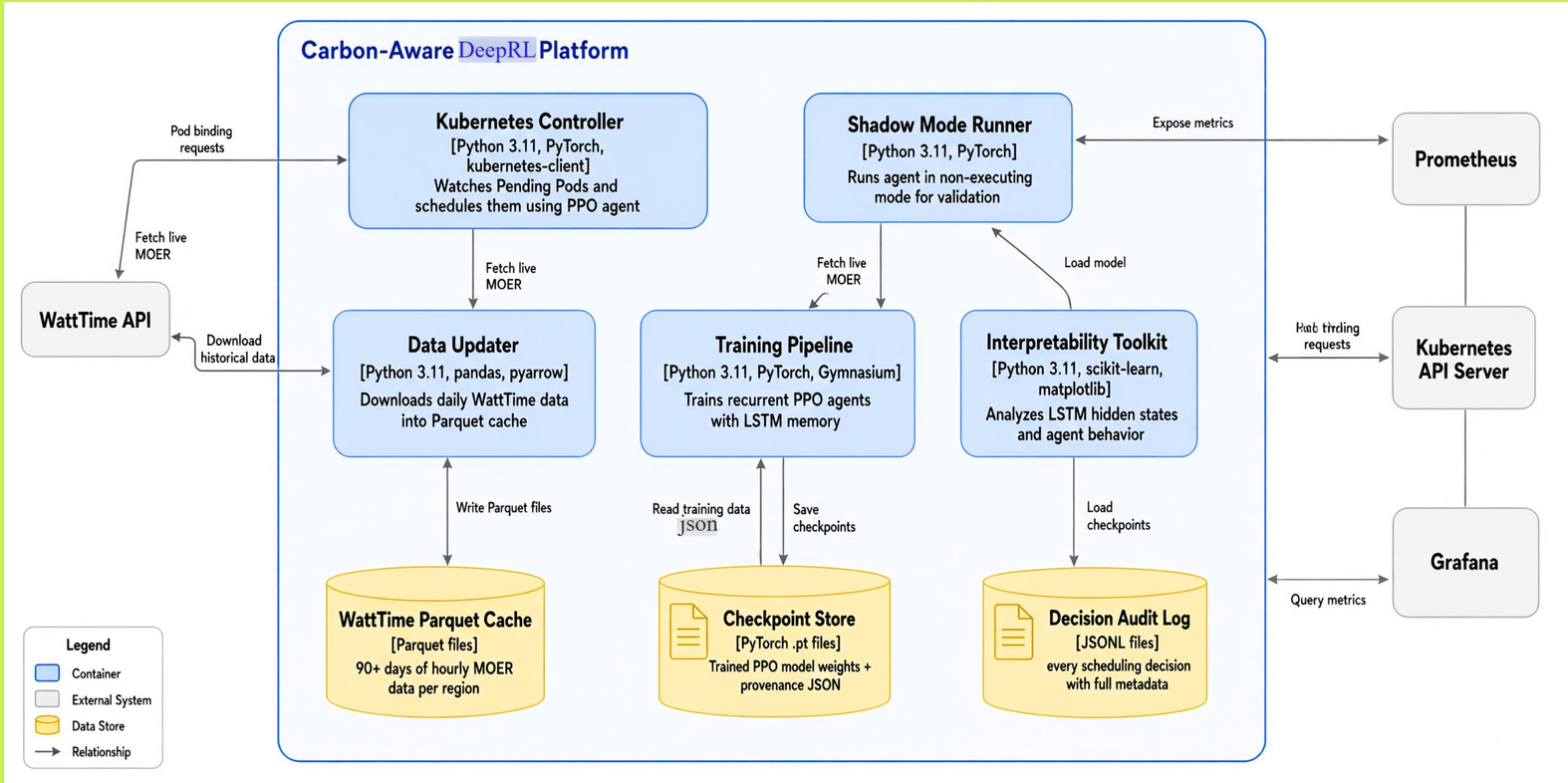
Trains and runs the LSTM-based PPO Agent

RUNTIME:

Shadow Mode for validation, Controller for execution

OBSERVABILITY:

Prometheus Metrics & Grafana Dashboards



The AI Engine - Recurrent PPO

The Agent doesn't just look at the current hour. It looks at a 5-hour rolling window. By using an LSTM, the agent learns to recognize patterns - like a diurnal solar curve - and anticipates future grid states implicitly, rather than relying on brittle external forecasts.

HOW THE RECURRENT PPO AGENT WITH LSTM MEMORY «THINKS» ?

Input (State)

33-dimensional vector

30 Dims

5-hour history Carbon Intensity & Utilization

3 Dims

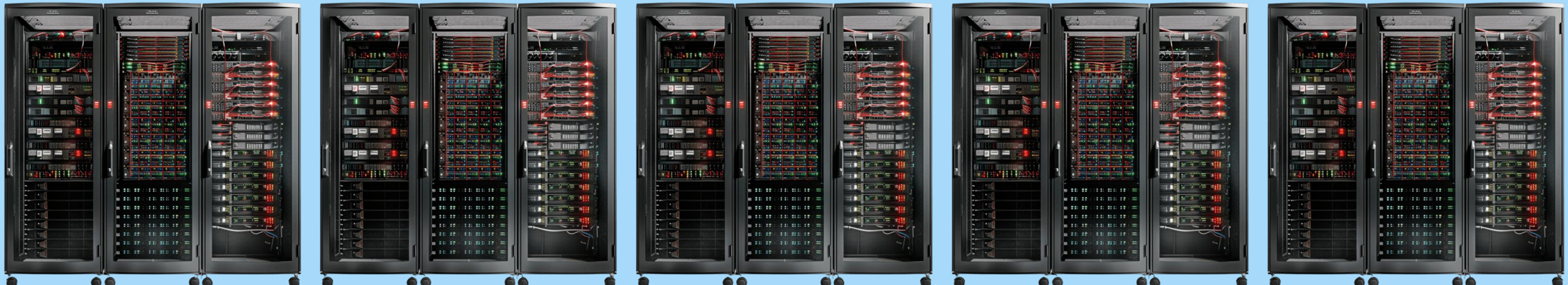
Current task features (Size, Time, SLA)

The Brain

LSTM network

Output (Action)

Probability distribution over available nodes



Safety First: Multi-Layered Risk Management

This is where we differ from academic prototypes. AI in production must be safe. We enforce risk management at three levels. We teach the AI to be risk-averse during training. We wrap it in a dynamic circuit breaker that shuts it down if it starts making bad decisions. And we require a mandatory shadow-mode validation period before it ever touches a real workload.

LAYER 1

Complete	Intrinsic (Training)
In Progress	Volatility Penalty: Agent is penalized for migrating during high grid instability.

LAYER 2

Complete	Extrinsic (Inference)
In Progress	Dynamic Circuit Breaker: Trips to «Safe Mode» if recent error/override rate >15%. Deterministic Safety Gate: Hard overrides for SLAs and capacity limits.

LAYER 3

Complete	Operational (Process)
In Progress	Mandatory Shadow Mode: 7-day non-executing validation period before production.

Gymnasium

Prometheus + Grafana



Proven Results on Real Grid Data



Experimental Method for Testing:

Holdout-Based Empirical Assessment of Mixed-Type Synthetic Data

Dataset (RTO USA):

90 days of real WattTime MOER data (California Independent System Operator, Midcontinent Independent System Operator, Electric Reliability Council of Texas).

Key Metrics (vs. Memoryless Baseline):

- 16.6% Reduction in Carbon Emissions ($p = 0.008$);
- 43% Fewer SLA Violations;
- 44% Faster Training Convergence.

Statistical Testing:

All experiments are run with 5 random seeds.

Mean ± Standard Deviation 95% Confidence Intervals;

Paired t-tests for Statistical Significance ($p < 0.05$).

Empirical Assessment of Mixed-Type Synthetic Data. Across five random seeds on real grid data, our recurrent agent reduced carbon emissions by 16.6% compared to a standard memoryless agent, with statistical significance. Furthermore, it actually reduced SLA violations, proving that temporal awareness leads to better, safer decisions.

Interpretability & Trust: Opening the «Black Box»

t-SNE Clustering:

Proves the LSTM separates «Clean» vs. «Dirty» grid regimes.

The LSTM (the AI's memory) processes the 5-hour grid history and compresses it into a 64-dimensional hidden state (a vector of 64 numbers). This is the AI's "internal understanding" of the current grid situation. t-SNE is a mathematical technique that takes these complex 64-dimensional vectors and squashes them down to 2 dimensions (X and Y coordinates) so we can plot them on a graph.

Temporal Attribution:

Shows the agent relies on recent history (t-1, t-2, t-3, t-4).

The AI looks at a 5-hour window of grid data (hours t-4, t-3, t-2, t-1, and t). But does it actually use all 5 hours equally? Or does it focus more on certain hours?Temporal Attribution (using a technique called occlusion) answers this by systematically "blinding" the AI to specific hours and seeing how much its decision changes.

Linear Probing:

Confirms the hidden state encodes 1-to-3 hour future predictions.

This is the most sophisticated test. We're asking: "Is the LSTM implicitly learning to predict the future, even though we never explicitly trained it to do so?" The LSTM is trained only to minimize carbon emissions, not to forecast. But if it's truly learning grid dynamics, its internal hidden state should contain information about what will happen in the next few hours.

Result:

High confidence-reward correlation (0.67).

Every time the AI makes a decision, it outputs: 1.An action: "Schedule on CAISO". 2.A confidence score: The probability it assigned to that action (e.g., 0.82 = 82% confident). Later, we observe the actual reward (negative carbon cost) that resulted from that decision. The confidence-reward correlation measures: "When the AI is confident, does it actually get better outcomes?" We collect thousands of (confidence, reward) pairs. Then we calculate the Pearson correlation coefficient.

Production Deployment & Observability



Deployment is seamless. It integrates directly with Kubernetes via a custom controller. Every single decision is logged with full provenance for audit purposes. Provide pre-built Grafana dashboards so operators can monitor carbon savings, agent confidence, and circuit breaker states in real-time.

MONITORING:

Prometheus metrics (14). Pre-built Grafana dashboards (System Health, Carbon Impact, Agent Behavior).

KUBERNETES INTEGRATION:

Custom Pod Controller with strict RBAC.

AUDITABILITY:

Every decision logged to JSONL with full provenance (Model hash, Git commit, Reason code).

45 ms

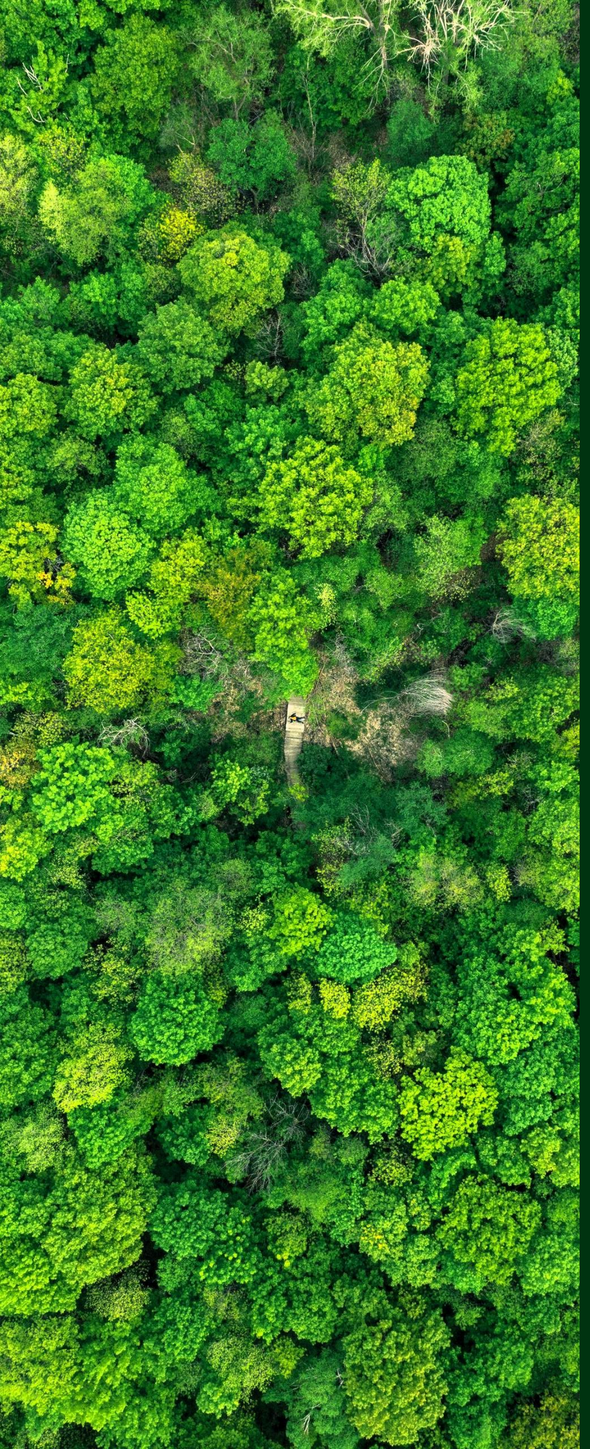
DECISION LATENCY (P99)

$R^2=0.48$

PREDICTIVE POWER (1-HR HORIZON)

15 %

CIRCUIT BREAKER TRIP THRESHOLD (ERROR RATE)



Thank You

Summary:

We have moved carbon-aware scheduling from a theoretical concept to a deployable, safe, and observable reality.

Future Avenues:

Explicit Forecast Integration (Predictive vs. Reactive).

Multi-Objective RL (Dynamic Carbon vs. Latency sliders).

Federated Learning (Cross-organization model training).

Hardware-Aware Scheduling (CPU vs. GPU power profiles).

